

```
render() {
  return (
    ...
    ...
  )
}
```

We can use the *useEffect* hook to do the exact same thing but with so much more ease! This is called ‘conditionally rendering’. To implement this, we pass a second argument to *useEffect*, i.e., an array of values that effect depends on. So, in this case, it’s the *count* and the effect runs only when the *count* changes. In this way, it gets easier to understand, and hooks simplify the code such that the three methods that are traditionally used are narrowed down to a single method, which behaves in the same way.

```
function MyCounter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  }, [count]);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

export default App;
```

useContext

```
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return (
    <button style={{ background: theme.background, color:
theme.foreground}}>
      I am styled by theme context!
    </button>
  );
}
```

The third basic hook is *useContext*. *useContext* accepts the context object and returns the current context value. The context object here is the value returned from *react.createContext* and it returns the current value. Basically, what it does is, it finds the nearest *MyContext.Provider* above the calling component in the React tree, and then it

reads its value. Whenever this value changes, *useContext* will trigger a re-render using the latest value of the context provider. But you should keep in mind that this hook is only used to read the context and subscribe to the changes.

useReducer

So, these were the basic hooks; now let’s look at the additional hooks. These hooks aren’t that commonly used but it’s good to know them and be aware of where to use each. *useReducer* is an alternative to *useState* and returns the current state paired with the dispatch method. So *useReducer* is generally preferable to *useState* when we have complex state logic -- usually when the next state depends on the previous one. It optimises performance for components as it passes down dispatch instead of the callbacks.

There are two ways we can initialise this state -- by specifying the initial state or by lazy initialisation. Specifying the initial state is the simplest way to visualise the state; just pass the state as the second argument to *useReducer*. In the code below, we set the *count* value to the initial value. Then this value can be defined in the implementation and passed as an object to *useReducer* as the second argument.

```
const [state, dispatch] = useReducer(
  reducer,
  {count: initialState}
);
```

You can also create the initial state lazily. To do this, we can define an *init* function that sets the initial count, and then pass this function as the third argument to *useReducer*. So the initial state will be set to *init*, and it lets you extract the logic for calculating the initial state outside the reducer.

```
function init(intitalCount) {
  return {count: initialState}
}
...
function Counter() {
  const [state, dispatch] = useReducer(reducer,
initialCount, init);
  return (
    <>
      Count: {state.count}
      <button
        onClick={() => dispatch({type: 'reset'})}>
        Reset
      </button>
    </>
  );
}
```